Software Design

Software Design is the process to transform the user requirements into some suitable form, which helps the programmer in software coding and implementation. During the software design phase, the design document is produced, based on the customer requirements as documented in the SRS document. Hence the aim of this phase is to transform the SRS document into the design document.

The following items are designed and documented during the design phase:

- Different modules required.
- Control relationships among modules.
- Interface among different modules.
- Data structure among the different modules.
- Algorithms required to implement among the individual modules.

Objectives of Software Design:

- Correctness: A good design should be correct i.e. it should correctly implement all the functionalities of the system.
- Efficiency: A good software design should address resources, time, and cost optimization issues.
- Understandability: A good design should be easily understandable, for which it should be modular and all the modules are arranged in layers.
- Completeness: The design should have all the components like data structures, modules, and external interfaces, etc.
- Maintainability: A good software design should be easily amenable to change whenever a change request is made from the customer side.

Software Design Concepts:

Concepts are defined as a principal idea or invention that comes into our mind or in thought to understand something. The software design concept simply means the idea or principle behind the design. It describes how you plan to solve the problem of designing software, the logic, or thinking behind how you will design software. It allows the software engineer to create the model of the system or software or product that is to be developed or built. The software design concept provides a supporting and essential structure or model for developing the right software. There are many concepts of software design and some of them are given below:

The following points should be considered while designing Software:

- Abstraction- hide Irrelevant data

Abstraction simply means to hide the details to reduce complexity and increases efficiency or quality. Different levels of Abstraction are necessary and must be applied at each stage of the design process so that any error that is present can be removed to increase the efficiency of the software solution and to refine the software solution.

- Modularity- subdivide the system

Modularity simply means dividing the system or project into smaller parts to reduce the complexity of the system or project. In the same way, modularity in design means subdividing a system into smaller parts so that these parts can be created independently and then use these parts in different systems to perform different functions.

- Architecture- design a structure of something

Architecture simply means a technique to design a structure of something. Architecture in designing software is a concept that focuses on various elements and the data of the structure. These components interact with each other and use the data of the structure in architecture.

- Refinement- removes impurities

Refinement simply means to refine something to remove any impurities if present and increase the quality. The refinement concept of software design is actually a process of developing or presenting the software or system in a detailed manner that means to elaborate a system or software. Refinement is very necessary to find out any error if present and then to reduce it.

- Pattern- a repeated form

The pattern simply means a repeated form or design in which the same shape is repeated several times to form a pattern. The pattern in the design process means the repetition of a solution to a common recurring problem within a certain context.

- Information Hiding- hide the information

Information hiding simply means to hide the information so that it cannot be accessed by an unwanted party. In software design, information hiding is achieved by designing the modules in a manner that the information gathered or contained in one module is hidden and can’t be accessed by any other modules.

- Refactoring- reconstruct something

Refactoring simply means reconstructing something in such a way that it does not affect the behavior of any other features. Refactoring in software design means reconstructing the design to reduce complexity and simplify it without affecting the behavior or its functions. Fowler has defined refactoring as “the process of changing a software system in a way that it won’t affect the behavior of the design and improves the internal structure”.

Structured Analysis and Structured Design (SA/SD)

Structured Analysis and Structured Design (SA/SD) is a diagrammatic notation that is designed to help people understand the system. The basic goal of SA/SD is to improve quality and reduce the risk of system failure. It establishes concrete management specifications and documentation. It focuses on the solidity, pliability, and maintainability of the system.

The method is based on the principle of structured programming, which emphasizes the importance of breaking down a software system into smaller, more manageable components.

In SA/SD, the software development process is divided into two phases: Structured Analysis and Structured Design. During the Structured Analysis phase, the problem to be solved is analyzed and the requirements are gathered. The Structured Design phase involves designing the system to meet the requirements that were gathered in the Structured Analysis phase.

Structured Analysis and Structured Design (SA/SD) is a traditional software development methodology that involves a series of techniques for designing and developing software systems in a structured and systematic way.

SA/SD is combined known as SAD and it mainly focuses on the following 3 points:

- System
- Process
- Technology

SA/SD involves 2 phases:

- Analysis Phase: It uses Data Flow Diagram, Data Dictionary, Decision Tables, State Transition diagram and ER diagram.
- Design Phase: It uses Structure Chart and Pseudo Code.

Data Flow Diagram

- Provides an indication of how data is transformed as it moves through the system
- Depicts the functions and sub-functions that transform data
- Start with the context diagram
- Decompose the system into 5-7 sub-processes
- Level 0 diagram depicts the 5-7 sub-process and the data stores used by the sub processes
- Check for consistency with the context diagram
- Each sub-process is then decomposed into sub-functions which are then diagramed
- For each process bubble on any diagram there is an accompanying PSPEC depicting detailed logic

Steps in Developing a Data Flow Diagram

- establish conceptual view – identify the outputs and their sinks – identify the inputs and their sources
- define top level processes – 5-7 sub functions – where data is transformed - not stored
- diagram each top level process
- define data stores
- build single diagram -- level 0

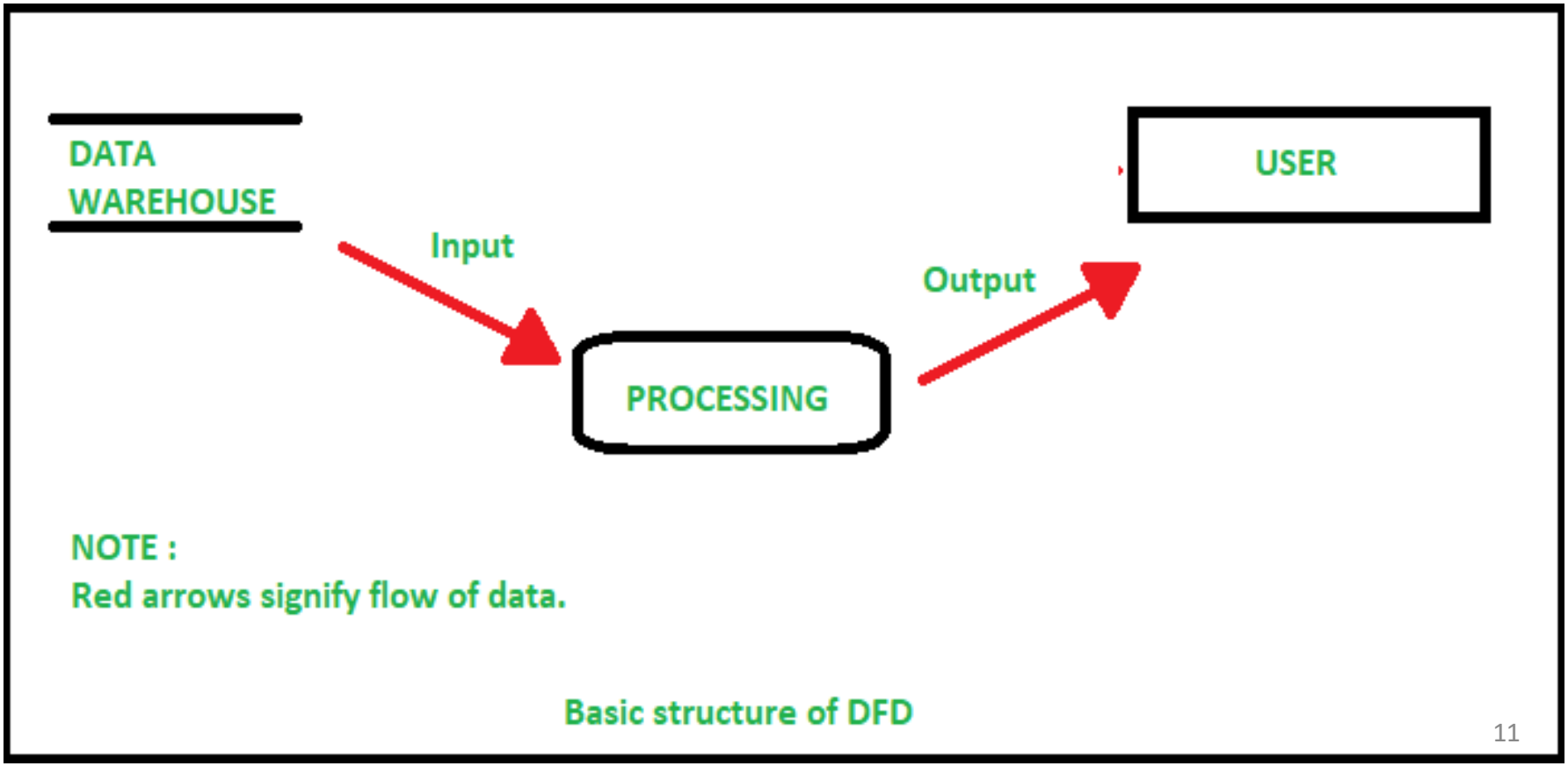
Defining the Top Level Processes

- determine the events that drive the system
  - examine each external entity and ask what effects their actions have on the system
  - examine each event and determine if all instances of the event involve the same data
  - event lists must include normal and abnormal processing Consistency

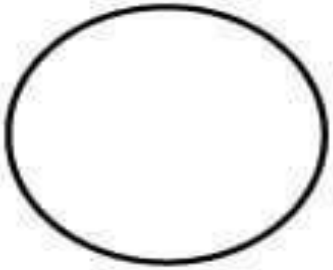
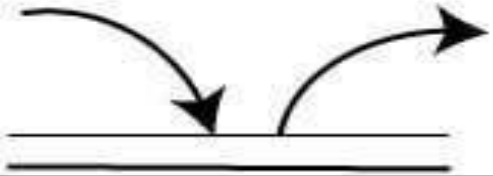
Checking with the Context Diagram

– each input flow is needed by the system to recognize that an event has occurred, or is needed to produce a response to an event

– each output flow is the response to an event – each non temporal event should have input from which the system can detect that the event has occurred (control events)



Basic structure of DFD

Symbol	Name	Function
	Data flow	Used to Connect Processes to each , other , to sources or Sinks; te arrow head indicates direction of data flow.
	Process	Perfroms Some transformation of Input data to yield output data.
	Source of Sink (External Entity)	A Source of System inputs or Sink of System outputs.
	Data Store	A repository of data; the arrow heads indicate net inputs and net outputs to store.

Symbols for Data Flow Diagrams

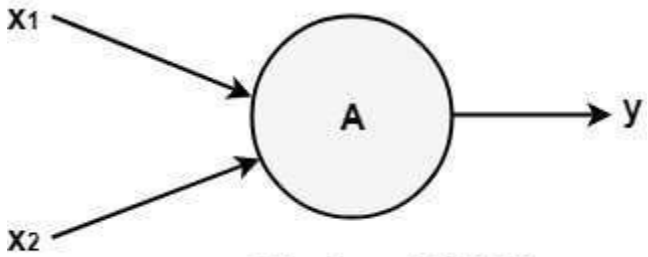


Fig: Level-0 DFD.

The Level-0 DFD, also called context diagram of the result management system is shown in fig. As the bubbles are decomposed into less and less abstract bubbles, the corresponding data flow may also be needed to be decomposed.

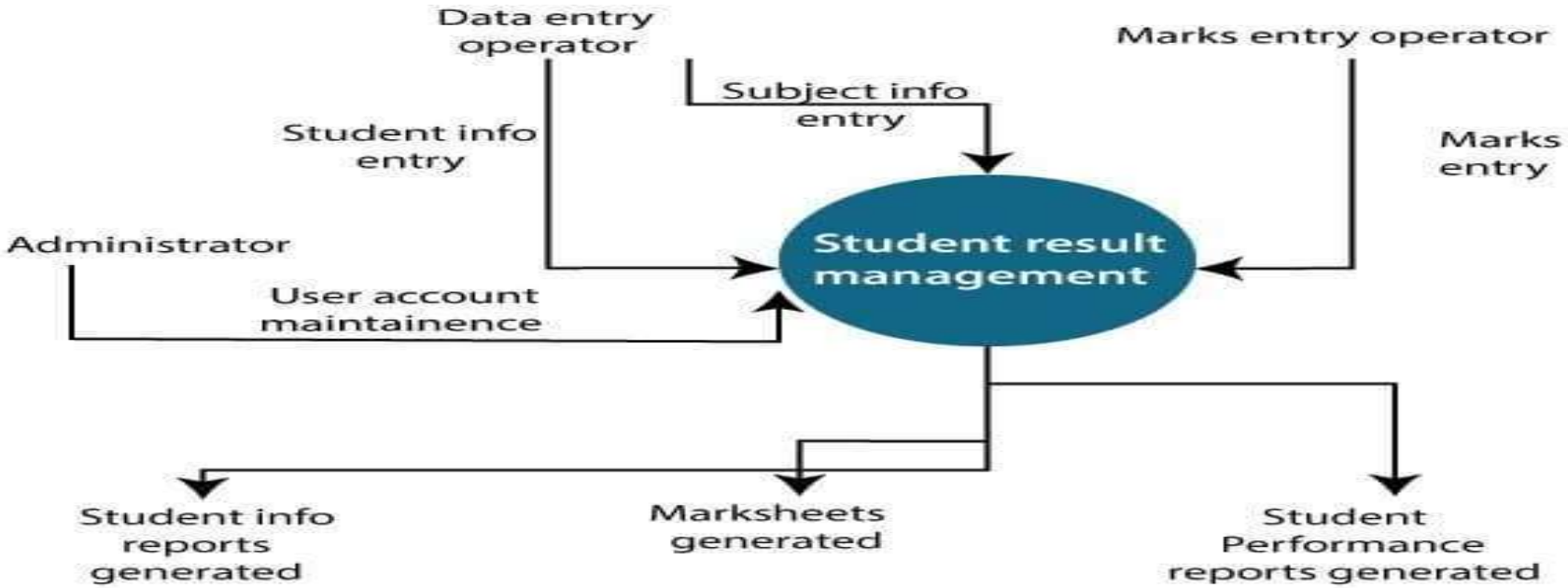


Fig: Level-0 DFD of result management system

1-level DFD

In 1-level DFD, a context diagram is decomposed into multiple bubbles/processes. In this level, we highlight the main objectives of the system and breakdown the high-level process of 0-level DFD into subprocesses.

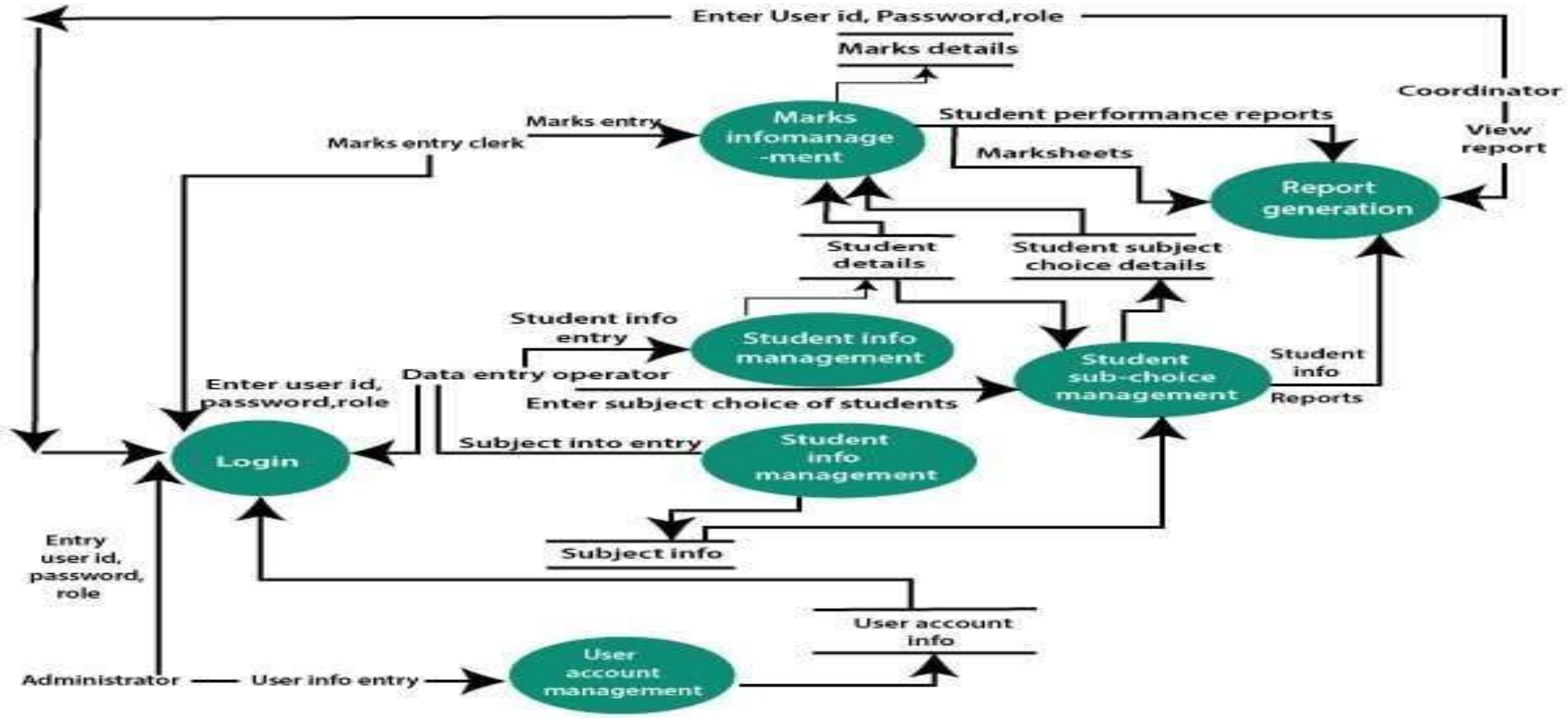
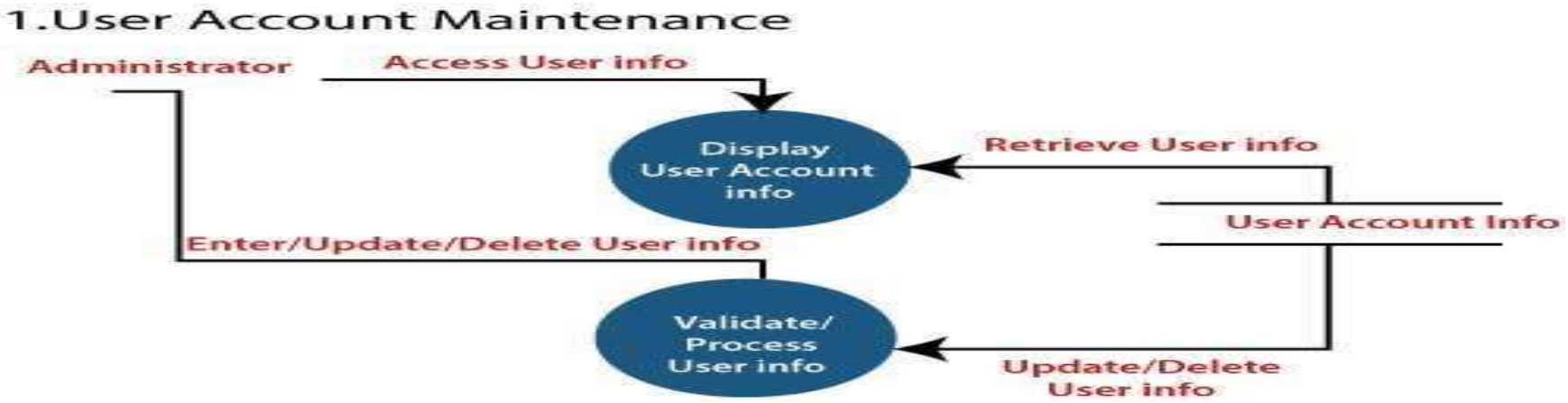
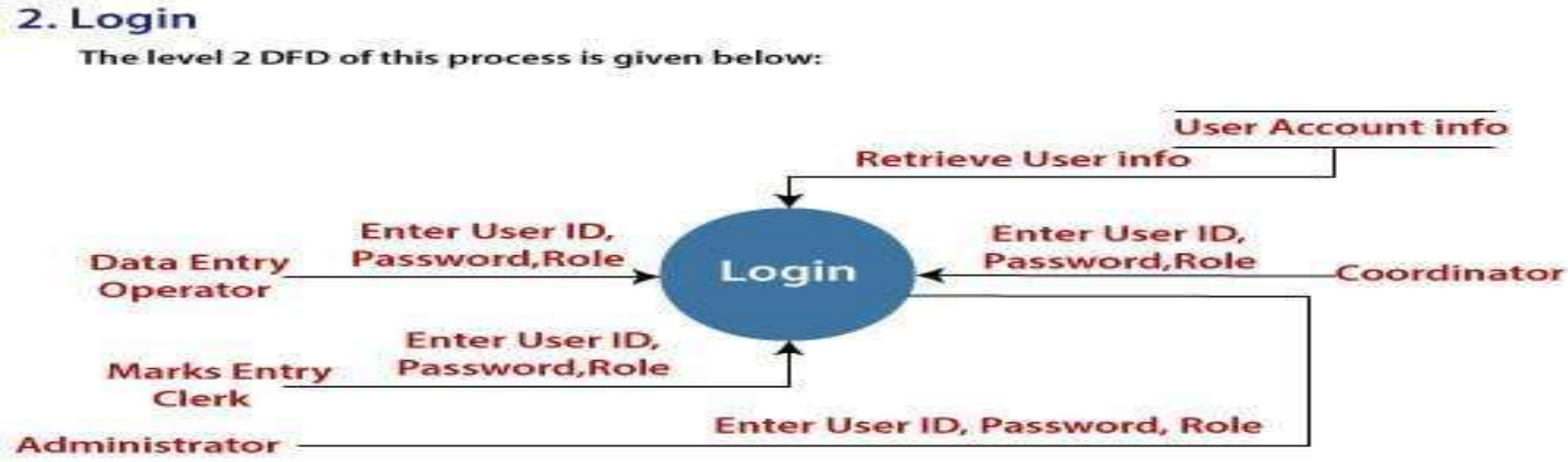


Fig: Level-1 DFD of result management system

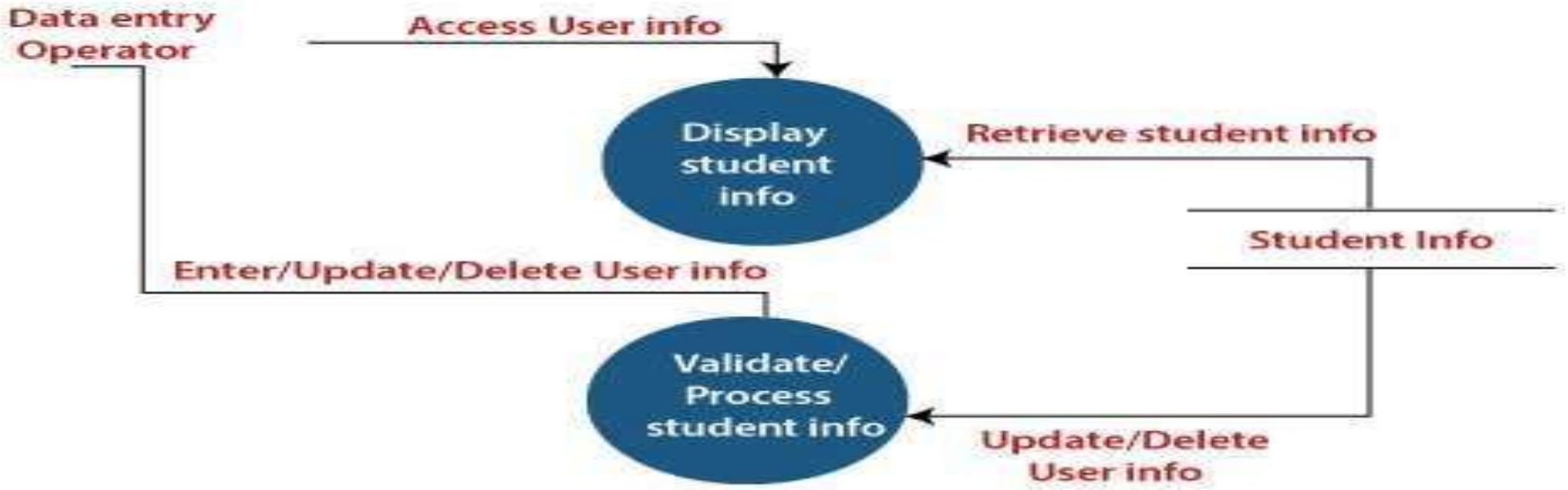
2-Level DFD

2-level DFD goes one process deeper into parts of 1-level DFD. It can be used to project or record the specific/necessary detail about the system's functioning.



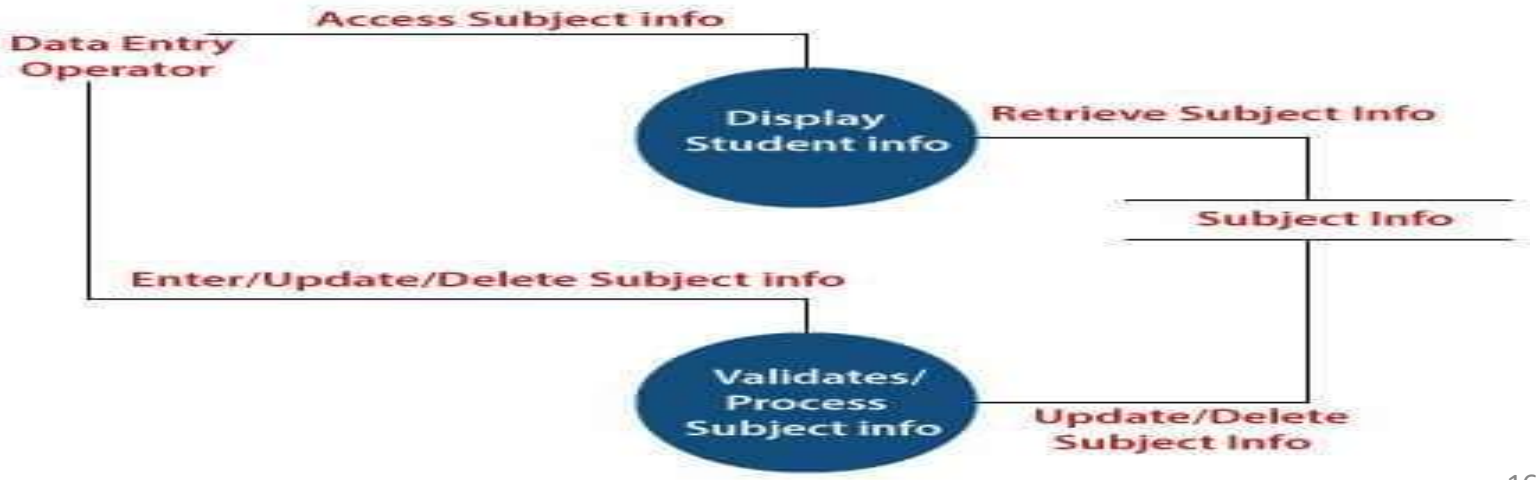


3. Student Information Management



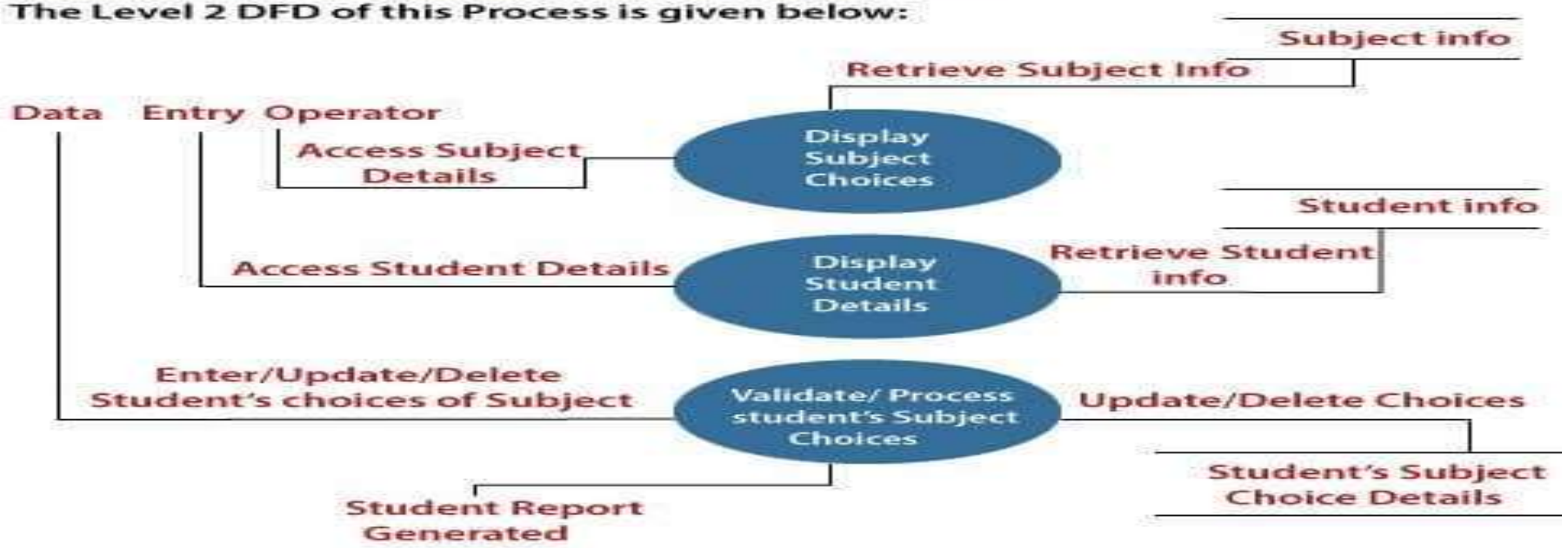
4. Subject Information Management

The level 2 DFD of this process is given below:



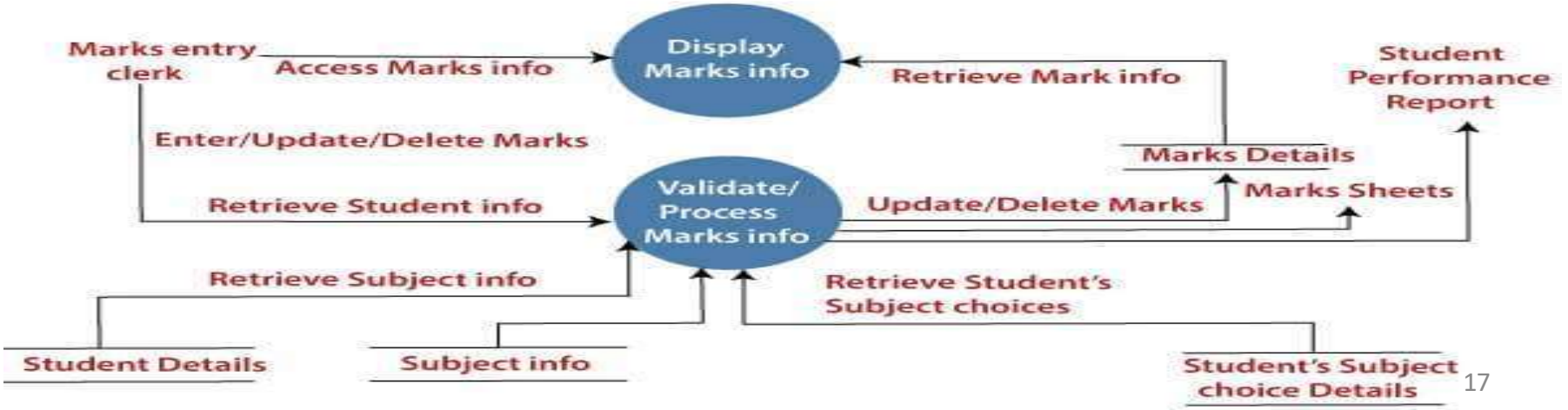
5. Student's Subject Choice Management

The Level 2 DFD of this Process is given below:



6. Marks Information Managment

The Level 2 DFD of this Process is given below:



Data Dictionary (DD)

A Data Dictionary can be defined as a collection of information on all data elements or contents of databases such as data types, and text descriptions of the system. It makes it easier for users and analysts to use data as well as understand and have common knowledge about inputs, outputs, components of a database, and intermediate calculations.

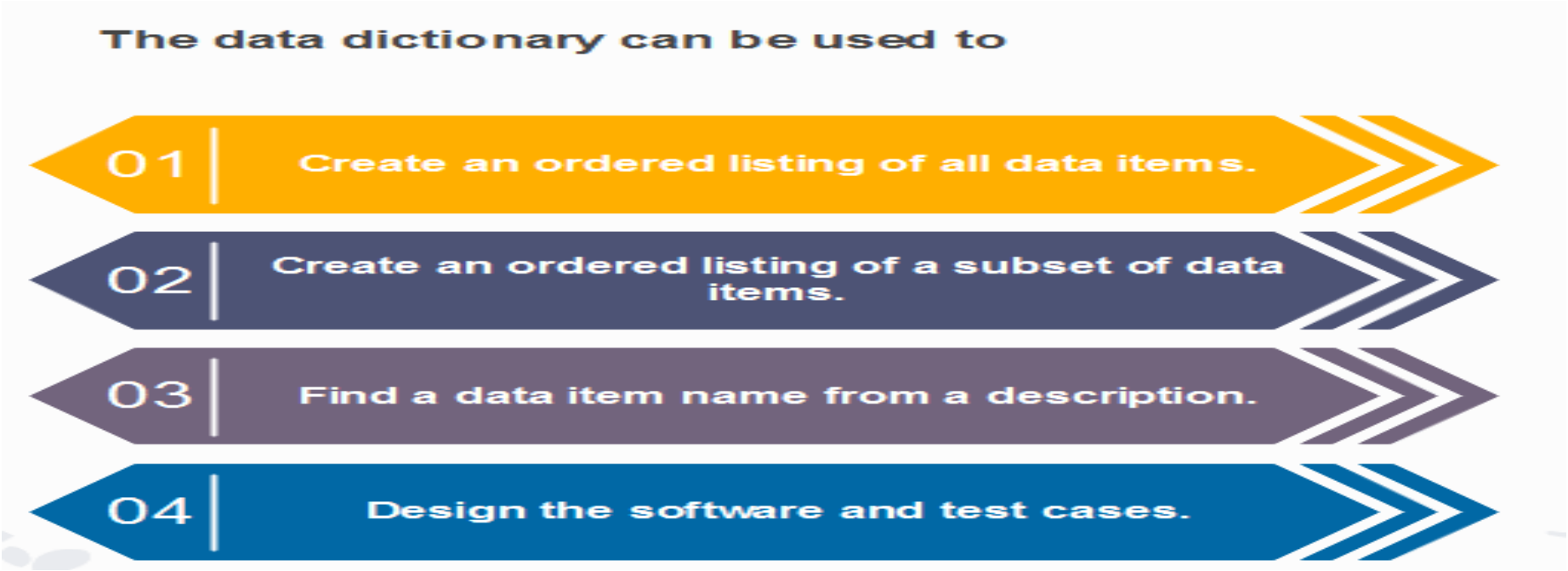
Why Data Dictionary is Essential?

There is less information and details provided by data models. So, a data dictionary is essential and needed to have proper knowledge and usage of contents. Data Dictionary provides all information about names that are used in system models. Data Dictionary also provides information about entities, relationships, and attributes that are present in the system model. As a part of the structured analysis and design tool, the implementation of a data dictionary is done.

The data dictionary, in general, includes information about the following:

- **Name of the data item**
- **Aliases**
- **Description/purpose**
- **Related data items**
- **Range of values**
- **Data structure definition/Forms**

- The name of the data item is self-explanatory.
- Aliases include other names by which this data item is called DEO for Data Entry Operator and DR for Deputy Registrar.
- Description/purpose is a textual description of what the data item is used for or why it exists.
- Related data items capture relationships between data items e.g., total_marks must always equal to internal_marks plus external_marks.
- Range of values records all possible values, e.g. total marks must be positive and between 0 to 100.
- Data structure Forms: Data flows capture the name of processes that generate or receive the data items. If the data item is primitive, then data structure form captures the physical structures of the data item. If the data is itself a data aggregate, then data structure form capture the composition of the data items in terms of other data items.



The mathematical operators used within the data dictionary are defined in the table:

Notations	Meaning
x=a+b	x includes of data elements a and b.
x=[a/b]	x includes of either data elements a or b.
x=a x	includes of optimal data elements a.
x=y[a]	x includes of y or more occurrences of data element a
x=[a]z	x includes of z or fewer occurrences of data element a
x=y[a]z	x includes of some occurrences of data element a which are between y and z.

Example: Let us understand this by taking an example of the reservation system. In the reservation system, “passenger” is a data item whose information is available on the data dictionary as follows:

Keys	Values
Name	Passenger
Aliases	None
Where or how it’s used?	Passenger’s query (input) Ticket (output)
Description	•Passenger = Passenger_name + Passenger_address •Passenger_name = Passenger_lastname + Passenger_firstname + Passenger_middle_initial •Passenger_address = Local_address + Community_address + Zip_code •Local_address = House_number + street_name + Apartment_number •Community_Address = City_name + State_name

The Make-Buy Decision or Decision Table

A decision table is a brief visual representation for specifying which actions to perform depending on given conditions. It is a good way to settle different combination inputs with their corresponding outputs and is also called a cause-effect table. It consists of rows and columns, separated into four quadrants. The upper left quadrant contains the condition(s); the upper right quadrant contains the condition alternatives. The lower half of the table contains the actions to be taken on the left and the rules for executing the actions on the right. When a decision table is used to determine which action needs to be taken, the logic moves clockwise beginning from the upper left.

Conditions and Actions	Rules
Conditions	Condition Alternatives
Actions	Action Entries

Suppose a store wanted to illustrate its policy on noncash customer purchases. The company could do so using a simple decision table as shown below.

Conditions and Actions	Rule 1	Rule 2	Rule 3	Rule 4
Under $50	Y	Y	N	N
Pays by check with two forms of ID	Y	N	Y	N
Uses credit card	N	Y	N	Y
Complete the sale after verifying signature.	X			
Complete the sale. No signature needed.		X		
Call supervisor for approval.			X	
Communicate electronically with bank for credit card authorization.				X

Each of the three conditions (sale under $50, pays by check, and uses credit cards) has only two alternatives. The two alternatives are Y (yes, it is true) or N (no, it is not true). Four actions are possible:

- Complete the sale after verifying the signature.
- Complete the sale. No signature needed.
- Call the supervisor for approval.
- Communicate electronically with the bank for credit card authorization.

The final ingredient that makes the decision table worthwhile is the set of rules for each of the actions. Rules are the combinations of the condition alternatives that precipitate an action. For example, Rule 3 says:

IF	**N**	**(the total sale is NOT under $50.00) AND**
IF	Y	(the customer paid by check and had two forms of ID) AND
IF	N	(the customer did not use a credit card) THEN
DO	X	(call the supervisor for approval).

Developing Decision Tables

To build decision tables, the analyst needs to determine the maximum size of the table; eliminate any impossible situations, inconsistencies, or redundancies; and simplify the table as much as possible. The following steps provide the analyst with a systematic method for developing decision tables:

- Determine the number of conditions that may affect the decision. The number of conditions becomes the number of rows in the top half of the decision table.
- Determine the number of possible actions that can be taken. That number becomes the number of rows in the lower half of the decision table.

- Determine the number of condition alternatives for each condition. In the simplest form of decision table, there would be two alternatives (Y or N) for each condition.
- Calculate the maximum number of columns in the decision table by multiplying the number of alternatives for each condition. If there were four conditions and two alternatives (Y or N) for each of the conditions, there would be 16 possibilities.
- Fill in the condition alternatives. Start with the first condition and divide the number of columns by the number of alternatives for that condition.
- Complete the table by inserting an X where rules suggest certain actions.
- Combine rules where it is apparent that an alternative does not make a difference in the outcome.
- Check the table for any impossible situations, contradictions, and redundancies. They are discussed in more detail later.
- Rearrange the conditions and actions (or even rules) if it makes the decision table more understandable.

Quality

The American Heritage Dictionary defines quality as “a characteristic or attribute of something.” As an attribute of an item, quality refers to measurable characteristics— things we are able to compare to known standards such as length, color, electrical properties, and malleability.

Nevertheless, measures of a program’s characteristics do exist. These properties include cyclomatic complexity, cohesion, number of function points, lines of code, and many others. When we examine an item based on its measurable characteristics, two kinds of quality may be encountered: quality of design and quality of conformance.

- **Quality of design** refers to the characteristics that designers specify for an item. The grade of materials, tolerances, and performance specifications all contribute to the quality of design. As higher-grade materials are used, tighter tolerances and greater levels of performance are specified, the design quality of a product increases, if the product is manufactured according to specifications.

- **Quality of conformance** is the degree to which the design specifications are followed during manufacturing. Again, the greater the degree of conformance, the higher is the level of quality of conformance.

User satisfaction = compliant product

+

good quality

+

delivery within budget and schedule

Software quality attributes

Software Quality attributes help to measure the quality of the software from different angles. Software Quality Attributes can be broadly classified into 5 types: Design, Runtime, System, User, and Non-runtime qualities.

- Reliability

Reliability is the ability of software applications to behave as expected and function under the maximum possible load. Reliability is made up of different types

- Availability: The percentage of time the application is available for use.

Recoverability: Recoverability refers to the ability to recover the system quickly and efficiently.

- Fault Tolerance:

This attribute refers to the extent to which the system can tolerate hardware failures.

- Maintainability

Maintainability refers to how easily software developers can add new features and update existing features with new technologies.

- Usability

The Usability attributes refer to the end user's ease of use. Usability is tied to application performance, application UX design, and accessibility.

- Portability

The portability quality attribute refers to how easily the system can be ported or migrated to other environments containing different hardware or operating system specifications.

- Correctness

Correctness refers to the ability to behave or function as per software requirement specifications.

- Efficiency

Efficiency can be defined as the time taken by the system to complete a specific task.

- Security

Security attribute focuses on the ability to safeguard applications, data, and information from unauthorized entities. Authentication, authorization, and data encryption are a few key areas that can safeguard from malicious attacks.

- Testability

Testability is how easily QA members can test the software and log a defect and how easy it is to automate the software applications.

- Flexibility

Flexibility refers to how quickly your application can adapt to future and current technology demands.

- Scalability

Scalability is how easily your system can handle increasing demands without affecting the application's performance.

- Compatibility

Compatibility focuses on the ability to work software systems on different operating systems, and browsers seamlessly as expected without affecting any functionality.

- Supportability

It is the degree to which a software system can provide useful information for identifying and resolving the issues when application/functionality stops working.

- Reusability

It is the degree to which software components can be reused in another application or the same application. Reusable software components reduce the development cost and effort.

- Interoperability

Interoperability refers to the ability to communicate or exchange data between different systems.

Factors Affecting Software Quality

The various factors, which influence the software, are termed as software factors. They can be broadly divided into two categories. The first category of the factors is of those that can be measured directly such as the number of logical errors, and the second category clubs those factors which can be measured only indirectly. For example, maintainability but each of the factors is to be measured to check for the content and the quality control.

Several models of software quality factors and their categorization have been suggested over the years. The classic model of software quality factors, suggested by McCall, consists of 11 factors (McCall et al., 1977).

McCall's Factor Model

This model classifies all software requirements into 11 software quality factors. The 11 factors are grouped into three categories – product operation, product revision, and product transition factors.

Product operation factors − Correctness, Reliability, Efficiency, Integrity, Usability.

Product revision factors − Maintainability, Flexibility, Testability.

Product transition factors − Portability, Reusability, Interoperability.

Software Quality Management

Software Quality Management ensures that the required level of quality is achieved by submitting improvements to the product development process. SQA aims to develop a culture within the team and it is seen as everyone's responsibility.

Software Quality management should be independent of project management to ensure independence of cost and schedule adherences. It directly affects the process quality and indirectly affects the product quality.

Activities of Software Quality Management:

- **Quality Assurance -** QA aims at developing Organizational procedures and standards for quality at Organizational level.
- **Quality Planning -** Select applicable procedures and standards for a particular project and modify as required to develop a quality plan.
- **Quality Control -** Ensure that best practices and standards are followed by the software development team to produce quality products.

Software Quality Assurance

Software quality assurance (SQA) is a process that assures that all software engineering processes, methods, activities, and work items are monitored and comply with the defined standards. These defined standards could be one or a combination of any like ISO 9000, CMMI model, ISO15504, etc.

SQA incorporates all software development processes starting from defining requirements to coding until release. Its prime goal is to ensure quality.

Software Quality Assurance Plan

Abbreviated as SQAP, the software quality assurance plan comprises the procedures, techniques, and tools that are employed to make sure that a product or service aligns with the requirements defined in the SRS(software requirement specification).

The plan identifies the SQA responsibilities of a team, and lists the areas that need to be reviewed and audited. It also identifies the SQA work products.

Building SQA Plan

The SQA plan document consists of the below sections:

- Purpose section
- Reference section
- Software configuration management section
- Problem reporting and corrective action section
- Tools, technologies, and methodologies section
- Code control section
- Records: Collection, maintenance, and retention section
- Testing methodology

Steps to Develop Software Quality Assurance Plan

Developing a Quality Assurance Plan (QAP) involves a systematic process to ensure that the plan effectively addresses the quality requirements of a project or process.

- Define Project Objectives and Scope:

Clearly articulate the objectives and scope of the project or process for which the Quality Assurance Plan is being developed. Understand the goals, deliverables, and stakeholders involved.

- Identify Quality Standards and Criteria:

Determine the relevant quality standards, regulations, and criteria that are applicable to the project. This may include industry standards, legal requirements, and internal organizational standards.

- Identify Stakeholders:

Identify and involve key stakeholders who have an interest in or will be affected by the quality of the project or process. This includes project managers, team members, customers, and any regulatory bodies.

- Define Roles and Responsibilities:

Clearly define the roles and responsibilities of individuals or teams involved in quality assurance activities. This ensures accountability and clarity in the execution of quality-related tasks.

- Conduct a Risk Assessment:

Identify potential risks to the quality of the project. Assess the impact and likelihood of these risks and develop strategies for risk mitigation and management.

- Establish Quality Assurance Activities:

Determine the specific activities and tasks that will be carried out to ensure quality throughout the project lifecycle. This may include reviews, inspections, audits, testing, and other quality control measures.

- Develop Testing and Inspection Procedures:

If applicable, create detailed testing and inspection procedures. This includes developing test plans, test cases, and acceptance criteria to ensure that the product or service meets quality standards.

- Document Processes and Procedures:

Document the processes and procedures that will be followed to implement quality assurance activities. This documentation serves as a reference for team members and provides a basis for consistency.

- Establish Documentation and Reporting Mechanisms:

Define the documentation requirements for tracking and reporting quality-related information. This may include reports, checklists, and records of quality assessments.

- Allocate Resources and Training:

Specify the resources, tools, and training that will be provided to team members to support quality assurance efforts. Ensure that team members have the necessary skills and knowledge.

- Define Change Control Processes:

Establish a process for managing changes to the project or product to ensure that they do not negatively impact quality. This includes a change control board and a structured process for reviewing and approving changes.

- Review and Approval:

Seek input and feedback from relevant stakeholders on the draft Quality Assurance Plan. Revise the plan based on feedback and obtain formal approval before implementation.

- Monitoring and Continuous Improvement:

Continuously monitor quality metrics, track progress, and identify opportunities for improvement. Regularly review and update the Quality Assurance Plan to adapt to changes and enhance its effectiveness.

- Communication and Training:

Communicate the Quality Assurance Plan to all relevant stakeholders and provide training as necessary. Ensure that everyone involved understands their roles and responsibilities in maintaining and improving quality.

Components of SQA Plan :

1. Pre-project components: –

This stage is characterized ahead of executing the undertaking. A workout which are few are done at this time, they are:

- Guaranteeing the assets, plan plus the investing plan required. Those need to characterize to synchronize sufficiently everything since it arranged.
- Apart from the assets, plan together with a spending plan, there are other plans that should be characterized. These are typically quality and improvement plans.
- The engineer plus the proprietor of this undertaking might have an alternative perspective concerning the quality. Therefore, it's characterized together by those two gatherings to modify the real points of view which will happen.

2. Components of project life cycle activities assessment: –

The workouts are partitioned into 2 phases at this time. They've been, the advancement life period phase and the activity upkeep stage. The workouts are acknowledging the master plan and development blunders during the enhancement life cycle phase. Also, it is likewise partitioned once again into the following sub-classes, that is:

- Reviews
- Expert opinions
- Software testing

Additionally, for the game support stage, the workouts that remember for that stage is the upkeep that is specific, since it's constantly done being developed life period phase. This movement is achieved for enhancing the usefulness of support projects. Other than that, there's a subclass that is extra of. It's to make sure the type of endeavor components performed by subcontractors and other members which are external task advancement and maintenance.

3. Components of infrastructure error prevention and improvement: –

The action that is fundamental of the stage would be to kill or make the mistakes disappeared or well, at any price reduce the rate of blunders. It's placed on the relationship that is entire.

4. Components of software quality management: –

The concept goal is controlling the enhancement maintenance and period and in addition, presenting the help that is administrative forestalling and limiting timetable and spending disappointment of the results.

5. Components of standardization, certification, and SQA system assessment: –

This phase executes an expert that is the global administrative norm for the association. The exercises remembered because of this stage are:

- Make use of the globally expert information
- Enhance the coordination of this quality that is respected with various associations
- Survey the achievement of value framework according to a scale that is typical

For the conventional, it's partitioned into 2 teams, they're:

- Quality administration standard
- Task measure standard

6. Organizing for SQA — the human components: –

At this phase, we're piecing together the individuals who relate to SQA things. They are supervisors, trying staff, SQA trustees, SQA panel individuals, and SQA gathering people. Each of them adds to start and support the execution of SQA parts, distinguish deviations from SQA strategy and approach, and propose upgrades.

Quality management principles

- Customer focus

As the title suggests, this aim is to focus on both direct and indirect customers. Your goal is to meet customer requirements and exceed them. The focus is on netting sustained success by focusing on customers' needs. Providing value-added services at every stage is vital. Customer focus means you need to manage customer relationships and watch customer satisfaction. Encourage your staff to talk to customers and find out what their needs are. You can then tailor your objectives to meet their needs.

- Leadership

The aim of leadership should be to establish unity of purpose. By doing this, your leaders improve effectiveness and efficiency within your company. Leadership comes down to communicating your business's quality objectives. Effective communication from leadership is vital. It aligns your company's vision, direction, and policies to achieve its aims. Share your commitment to quality with your people to inspire and encourage them. They'll then contribute to your ethos of quality.

- Engagement of people

The ISO 9001:2015 statement reads:

"Competent, empowered, and engaged people at all levels throughout the organisation are essential to enhance its capability to create and deliver value".

By engaging and empowering people at all levels, you encourage them to thrive. A thriving workforce adds value to their organisation's quality management. You can meet this goal by talking to your staff and affirming employees' contributions to your business.

- Process approach

Quality management systems consist of several different processes working together. Appoint people to develop and oversee these processes to be effective. Ensure that everybody understands the objectives and knows what the end result should be. Share information to help the continual improvement of your quality management. The process approach aims to provide consistent and predictable results. Consistency enhances customer confidence in your ability to deliver.

- Improvement

Focus on constant improvement by responding to market changes and customers' changing needs. Gear all your quality systems toward finding new ways to do things and make the services or goods you provide better. An organisation that loses the ability of continual improvement loses business. Avoid resistance to change. Something that worked 10 years ago may not be relevant today. Your organisation needs to evolve and improve to remain competitive.

- Evidence-based decision-making

Ensure that your people base their choices on the gauging of on-hand data and information. Make relevant data available to the decision-makers in your organisation, encouraging objectivity. Having access to reliable data allows for an improved decision-making process. Also, it helps with your company's capability to review and change decisions or opinions. It also allows you to determine past decisions' effectiveness. To make this possible, you need to train staff to study and test data and information that passes through their hands.

- Relationship management

ISO 9001:2015 focuses on relationship management with suppliers and partners. Expand this to include clients. The idea works on the theory that you need to have a good relationship with all interested parties. You need a good rapport with partners for them to supply you efficiently when activities overlap. By pooling information, expertise, and resources, you benefit all parties involved. A good relationship with suppliers results in an effective and efficient supply chain.

These seven principles constitute the baseline for QMS in any organization. Conduct internal audits to determine whether your company follows these principles. Remember that different activities and organizations have differing requirements. Decide on your company's core business needs and tailor the QMS to the circumstances. Test the salient data and information to decide on the shape of your quality management strategy. Understanding these principles helps in setting up a QMS that works for you.